

Code Optimization Techniques

Compiler Design (CS 3007)

Sumanta Pyne

Assistant Professor
Computer Science and Engineering Department
National Institute of Technology, Rourkela, INDIA

April 14, 2026

Constant folding

- $i = 320 * 200 * 32; \implies i = 2,048,000;$
- $y = 0 * x; \implies y = 0;$
- $S = \text{"abc"} + \text{"def"}; \implies S = \text{"abcdef"};$

Constant propagation

- `int x = 14;`
`int y = 7 - x / 2;`
`return y * (28 / x + 2);`
- Propagation of `x`
 - `int x = 14;`
`int y = 7 - 14 / 2;`
`return y * (28 / 14 + 2);`
- Further propagation with dead code elimination of both `x` and `y`
 - `int x = 14;`
`int y = 0;`
`return 0;`

Dead code elimination

- ```
int foo(void)
{
 int a = 24;
 int b = 25;
 int c;
 c = a * 4;
 return c;
 b = 24;
 return 0;
}
```
- Eliminate assignment to dead variable - `int b = 25;`
- Eliminate unreachable code
  - `b = 24;`  
`return 0;`

# Common subexpression elimination

- $A[l+1]=B[l+1];$ 
  - Common subexpression -  $l+1$
  - $J=l+1;$   
 $A[J]=B[J];$
- $a = b * c + g;$   
 $d = b * c * e;$ 
  - Common subexpression -  $b * c$
  - $tmp = b * c;$   
 $a = tmp + g;$   
 $d = tmp * e;$

# Loop-invariant code motion

- ```
int i = 0;
while (i < n) {
    x = y + z;
    a[i] = 6 * i + x * x;
    ++i;
}
```
- $x = y + z$ and $x * x$ is loop-invariant
- Precautions must be taken before moving the code outside the loop
- ```
int i = 0;
if (i < n) {
 x = y + z;
 int const t1 = x * x;
 do {
 a[i] = 6 * i + t1;
 ++i;
 } while (i < n);
}
```

# Induction variable elimination

- ```
int max = 10;
int result = 0;
for (int i = 0; i < max; i++) {
    result += 2;
}
return result;
```
- Basic induction variables - i and result
- Eliminate “i” by replacing its uses with “result”
- ```
int max = 10;
int result = 0;
for (; result < max*2; result+=2) {}
return result;
```

# Strength reduction

- ```
c = 7;  
for (i = 0; i < N; i++){  
    y[i] = c * i;  
}
```
- Multiplication of loop invariant c and induction variable i
- Replace with successive weaker additions
- ```
c = 7;
k = 0;
for (i = 0; i < N; i++){
 y[i] = k;
 k = k + c;
}
```

# Loop unrolling

- Normal loop
- ```
int x;  
for (x = 0; x < 100; x++){  
    delete(x);  
}
```
- After loop unrolling
- ```
int x;
for (x = 0; x < 100; x += 5){
 delete(x);
 delete(x + 1);
 delete(x + 2);
 delete(x + 3);
 delete(x + 4);
}
```
- With loop unrolling factor 5 new program require 20 iterations
- Branch penalty is minimized
- Increases program code size and register pressure

# Loop interchange

- Given code

```
for i from 0 to 10
 for j from 0 to 20
 a[j,i] = i + j
```
- After loop interchange

```
for j from 0 to 20
 for i from 0 to 10
 a[j,i] = i + j
```
- Improves locality of reference for multi-dimensional array
- Prevent cache misses

# Loop fusion/loop jamming

- Given code
- ```
int i, a[100], b[100];  
for (i = 0; i < 100; i++) {  
    a[i] = 1;  
}  
for (i = 0; i < 100; i++) {  
    b[i] = 2;  
}
```
- After loop fusion
- ```
int i, a[100], b[100];
for (i = 0; i < 100; i++) {
 a[i] = 1;
 b[i] = 2;
}
```
- Avoids the overhead of the loop control structures
- Achieves instruction-level parallelism
- Vice versa - loop fission/loop distribution

# Loop tiling/loop blocking/strip mine and interchange

- Loop nest optimization for
  - locality optimization
  - parallelization
  - nested loop overhead reduction
- Partitions a loop's iteration space into smaller chunks or blocks
- Ensure data used in a loop stays in the cache until it is reused
- for ( $i=0$ ;  $i<N$ ;  $++i$ ) {  
    ...  
}
- Blocked with a block size  $B$  by replacing it with
- for ( $j=0$ ;  $j<N$ ;  $j+=B$ ) {  
    for ( $i=j$ ;  $i<\min(N, j+B)$ ;  $++i$ ) {  
        ...  
    }  
}

# Loop tiling/loop blocking/strip mine and interchange

- Matrix-vector multiplication
- ```
int i, j, a[100][100], b[100], c[100];  
int n = 100;  
for (i = 0; i < n; i++) {  
    c[i] = 0;  
    for (j = 0; j < n; j++) {  
        c[i] = c[i] + a[i][j] * b[j];  
    }  
}
```

Loop tiling/loop blocking/strip mine and interchange

- Loop tiling using $2 * 2$ blocks
- ```
int i, j, x, y, a[100][100], b[100], c[100];
int n = 100;
for (i = 0; i < n; i += 2) {
 c[i] = 0;
 c[i + 1] = 0;
 for (j = 0; j < n; j += 2) {
 for (x = i; x < min(i + 2, n); x++) {
 for (y = j; y < min(j + 2, n); y++) {
 c[x] = c[x] + a[x][y] * b[y];
 }
 }
 }
}
```

- Optimal tiling size requires
  - accurate estimate of accessed array regions in the loop and
  - the cache size of the target machine